



INDIA.RUNS

Build what next India runs on

Team Name : Tech_Fusion

Team Leader Name : Koushik Ram Reddy

Problem Statement : AI-Powered Candidate Ranking System

Solution Overview

Our system ranks candidates by understanding genuine fit across 6 weighted signal dimensions.

Key Differentiators:

- Reads career narratives, not just skill tags (production vs. research)
- 3-Tier JD-derived skill taxonomy
- Behavioral signals (GitHub, response rate, interview completion)
- Gaussian experience curve matching JD sweet spot
- Multiplicative disqualifier penalty for pure-research/non-tech profiles
- Streaming architecture processes 100K candidates in 55 seconds on CPU

JD Understanding & Candidate Evaluation

Hard Requirements (Disqualify if missing):

- Production embedding retrieval (Sentence-Transformers, BGE, E5)
- Vector DB / hybrid search (FAISS, Milvus, Qdrant)
- Strong Python & 5-9 years experience

Candidate Signals Used:

- Skills (proficiency, duration, endorsements)
- Career history (titles, descriptions, industries)
- Platform signals (response rate, github activity, etc.)

Ranking Methodology

$$\text{Formula: final_score} = \sum (\text{weight_i} \times \text{signal_i}) \times (1 - \text{penalty})$$

Signal	Weight	What It Measures
Core AI/ML Skills Match	35%	Tier weight × proficiency × duration × log(endorsements)
Experience Fit	20%	Gaussian peak at 7 yrs ($\sigma=3$) + title + industry
Career Depth / Production	15%	Production keywords vs. research-only in descriptions
Platform Behavioral Signals	15%	Response rate, GitHub score, interview completion
Nice-to-Have Bonuses	10%	LoRA, LTR, HR-tech, OSS contributions
Disqualifier Penalty	$\times(1-p)$	Pure research, non-tech title, inactive profile

Explainability & Data Validation

Explainability (No Black Boxes):

- Every score is fully decomposed into 6 named signals.
- Output example: "Lead AI Engineer with 6.7 yrs; 14 AI/ML core skills; response rate 0.73..."

Data Validation & Anti-Gaming:

- No hallucinations — scores derived purely from structured JSON fields.
- No LLM in ranking loop — zero risk of fabricated qualifications.
- Endorsement trust uses $\log(1+n)$ to prevent keyword stuffing.

End-to-End Workflow

Step 1: JD Analysis

- Extract requirements and build 3-tier skill taxonomy.

Step 2: Streaming Candidate Processing

- Open candidates.jsonl line-by-line (no full RAM load).
- Compute 6 signal scores, apply penalty, maintain min-heap of top 100.

Step 3: Output Generation

- Sort heap, generate reasoning string, write CSV.



Results & Performance
Dataset: 100,000 candidates | Runtime: 55 seconds | Memory: <50 MB

Rank	Candidate ID	Title	Yrs	Skills	Score
1	CAND_0081846	Lead AI Engineer	6.7	14	0.990
2	CAND_0018499	Senior ML Engineer	7.2	13	0.934
3	CAND_0077337	Staff ML Engineer	7.0	12	0.904
4	CAND_0008425	Senior NLP Engineer	7.8	12	0.865
5	CAND_0071974	Senior AI Engineer	7.8	14	0.825
6	CAND_0086022	Senior Applied Scientist	5.3	16	0.817
7	CAND_0050876	Applied ML Engineer @ Freshworks	6.0	13	0.808
8	CAND_0002025	Senior AI Engineer @ Apple	5.9	15	0.793
9	CAND_0088025	Staff ML Eng @ Yellow.ai	8.6	15	0.740
10	CAND_0087630	AI Engineer	7.2	13	0.726

Core Language & Libraries:

- Python 3.14 — Standard library only (json, csv, heapq, math, re).
- Zero third-party dependencies for ranking → maximum portability.

Memory Architecture:

- Streaming JSONL reader + min-heap of 100 → handles 487 MB in $O(1)$ space.

Demo & Version Control:

- Streamlit (v1.35+) sandbox at india-runs-1.streamlit.app
- GitHub for version control (Clean history)

Submission Assets

GitHub Repository:

- <https://github.com/koushik2024-code/India-Runs>

Live Demo (Streamlit Sandbox):

- <https://india-runs-1.streamlit.app/>

Submission CSV:

- Tech_Fusion.csv — 100 ranked candidates, validated

Team — Tech_Fusion:

- Koushik Ram Reddy (Lead), Saiprakash Kulkarni, Md Shameem Alam, KANUBUDDI Bhanu



INDIA.RUNS
Thank You!

Team Tech Fusion
Build what next India runs on
India Runs Data & AI Challenge 2026

THANK YOU